

Chapter 10: Classes, Objects, and Methods: Object-Oriented Programming w/ Python

EEC 3150

Trevecca Nazarene University

1 Classes, Objects, and Methods

Classes, Objects, and Methods

Classes, Objects, and Methods

Data types vs. objects:

Definitions:

- A **data type** is a particular format for representing data of a certain kind
- An **object** is an particular instance of a data type

Examples:

- data type: `int`
object: `1`
- data type: `str`
object: `‘‘hello’’`
- data type: `builtin_function_or_method`
object: `print()`

Defn: classes and objects

A **class** is (a template for creating) an abstract data type

- We can create our own custom data types
- All Python data types are actually classes

An **object** is an particular instance of a class

Analogy:

- class: car
object: Honda Civic

Classes, Objects, and Methods

First class: Person

Define a Person class with a name attribute:

Defining the Person class:

```
class Person:
    def __init__(self, name):
        self._name = name
```

Creating a Person object:

```
p = Person('Graham')
```

Getting p's name:

```
p._name
```

- Create a class by using the `class` keyword, the name of the class, and a colon
- *method*: any function defined in the scope of a class
 - Every class should have a *constructor* method called `__init__` which takes `self` and any attributes as arguments
 - It is automatically called whenever a new class is created
 - The `self` variable is a reference to whatever particular object is calling a method; all methods definitions must have it, but nothing is ever passed to it when called
- *attribute*: any variable associated with a class
 - Set any attributes passed to the constructor
 - It is convention for all attributes to start with underscores

Classes, Objects, and Methods

First class: Person

Define a Person class with a name attribute:

Defining the Person class:

```
class Person:
    def __init__(self, name):
        self._name = name
```

Creating a Person object:

```
p = Person('Graham')
```

Getting p's name:

```
p._name
```

- To create a Person object, simply pass all necessary attributes to the Person function
 - Person is the class, p is the object
 - Again, nothing is passed for the self argument
- To access or change an object's attribute or call a method on it, use dot notation

Classes, Objects, and Methods

Person with multiple attributes and methods:

```
class Person:
    def __init__(self, name, age):
        self._name = name
        self._age = age
    def HaveBirthday(self):
        self._age += 1
```

```
p = Person('Bill', 50)
p.HaveBirthday()
```

- Person class has two attributes (`_name`, `_age`) and two methods (`__init__`, `HaveBirthday`)
- `HaveBirthday` will increment `_age` by 1

The `vars()` and `isinstance()` functions:

- `vars(x)` returns a dictionary of all attributes of the object `x`
- `isinstance(x, Class)` check whether the object `x` is an instance of the `Class` class

Rectangle class:

Create a Rectangle class with the following attributes

- length
- width

and the following methods

- get_area()
- get_perimeter()